

Artificial Intelligence

BS (CS) Spring 2026

Lab 12 Manual

Learning Objectives:

Supervised Learning – Complete Model Guide

Supervised Learning

Supervised learning is a type of machine learning where a model is trained using labeled data. In labeled data, each input is paired with a known output, allowing the model to learn patterns and relationships between them. This structured approach enables the model to make accurate predictions for new, unseen data.

As an example consider supervised learning as similar as guiding a student. The labeled data acts like a tutor, providing correct answers so the model (the student) can learn over time. Through repeated exposure to this labeled data, the model gradually becomes better at predicting the correct outcome on its own.

The main goal of supervised learning is to generalize from the labeled data so that it can make reliable predictions or decisions for unknown data in the future.

How Does Supervised Learning Work?

The process of supervised learning involves several key steps to train a model effectively:

- **Data Preparation** – The first step is preparing the data by cleaning and pre-processing it. This includes removing any irrelevant information, handling missing values, and organizing the data into labeled pairs, where each input has a corresponding output.
- **Training the Model** – The model is then trained using this labeled data. During training, the model analyzes the patterns in the input data and learns to map these inputs to their correct outputs. This involves adjusting the model's parameters to improve accuracy.
- **Model Evaluation** – Once trained, the model is tested on unseen data to evaluate its performance. This helps determine how well the model can generalize beyond the training data.

- **Making Predictions** – Finally, after the model has been trained and tested, it can be used to make predictions on new data. The better it learned during training, the more accurate its predictions will be.

Supervised Learning Algorithms

Supervised learning includes a variety of algorithms, each suited for different types of tasks. Here are the common supervised learning algorithms covered in this lab manual:

- **Decision Tree** – A tree-structured model that makes decisions by splitting data based on feature values. Highly interpretable and used for both classification and regression.
- **Linear Regression** – A simple algorithm that predicts continuous values by finding a linear relationship between input features and the target variable.
- **Logistic Regression** – Used for classification tasks. It predicts binary or multi-class outcomes using the sigmoid function to output probabilities.
- **K-Nearest Neighbors (KNN)** – Classifies or predicts based on the majority label or average value among the k closest data points.
- **Support Vector Machine (SVM)** – Creates an optimal hyperplane to separate classes. Effective for high-dimensional data.
- **Naive Bayes** – A probabilistic classifier based on Bayes' theorem. Very fast and effective for text classification.
- **Random Forest** – An ensemble of decision trees that reduces overfitting and improves accuracy through bagging and feature randomness.
- **Gradient Boosting** – An ensemble technique that builds models sequentially, each correcting errors of the previous. State-of-the-art accuracy.
- **Neural Networks (MLP)** – Inspired by the human brain, these models learn complex non-linear patterns, used in image and speech recognition.

Types of Supervised Learning

Supervised learning generally falls into two main types, each designed to handle different kinds of problems:

- **Classification** – The model predicts discrete categories or classes. For example, an email can be classified as "spam" or "not spam," or an image as "cat" or "dog." Classification is useful for tasks where outcomes fall into specific groups.
- **Regression** – The model predicts continuous values. For instance, predicting the price of a house based on features like location, size, and age. Regression is used when the output is a numeric range rather than distinct categories.

Algorithm Comparison Table

Algorithm	Type	Purpose	Method	Strengths	Use Cases
Decision Tree	Both	Model decisions & outcomes	Tree-like splits	Interpretable, no scaling	Classification & Regression
Linear Regression	Regression	Predict continuous values	Least squares linear fit	Simple, fast	Price prediction, forecasting
Logistic Regression	Classification	Predict binary/multi-class	Sigmoid function	Probabilistic output	Spam, disease detection
KNN	Both	Predict by proximity	k nearest neighbors	No training phase	Recommendation, recognition
SVM	Both	Optimal class separation	Maximizing margin hyperplane	High-dimensional data	Image classification, NLP
Naive Bayes	Classification	Probabilistic classification	Bayes' theorem	Very fast, text tasks	Spam filtering, sentiment
Random Forest	Both	Ensemble of trees	Bagging + feature randomness	Robust, high accuracy	Finance, healthcare
Gradient Boosting	Both	Sequential error correction	Iterative residual fitting	State-of-the-art accuracy	Competitions, ranking
Neural Networks	Both	Complex pattern learning	Backpropagation, layers	Handles complex patterns	Image, speech, NLP

Basic Structure

The main steps performed on any dataset in supervised learning generally include the following phases:

Data Collection

Gather relevant, representative, and sufficient data for the problem. The quality of data directly determines the quality of the learned model.

Load and Transform Dataset

Load the dataset from files (CSV, Excel, database) into a suitable data structure such as a Pandas DataFrame. Apply initial transformations such as parsing dates, renaming columns, or merging tables.

Data Exploration and Analysis

Understanding of the relevant dataset through exploratory analysis.

Exploratory Data Analysis (EDA)

Understand the dataset by summarizing its main characteristics, often using visual methods such as histograms, box plots, scatter plots, and correlation heatmaps.

Statistical Analysis

Calculate descriptive statistics (mean, median, mode, standard deviation) to understand the data distribution.

- **Statistical Summary:** Use `.describe()` and `.info()` to get an overview of all features.
- **Skewness and Kurtosis Check:** Skewness and kurtosis describe the asymmetry and tail behavior of data, helping to detect outliers and check whether data meets assumptions for statistical analysis and machine learning models.
- **Distribution of Engagement Features:** Distribution of engagement features helps us understand user behavior patterns, detect anomalies, and ensure the data is suitable for accurate modeling and analysis.
- **Correlation Analysis:** Measures the strength and direction of relationships between variables. We want high correlation between independent features and the dependent variable (useful predictors) and low correlation between independent features themselves (to avoid multicollinearity).
- **Outlier Detection:** The process of identifying data points significantly different from most of the data, as they can distort analysis and reduce model accuracy.
- **Z-Score:** Measures how many standard deviations a data point is from the mean, used to detect outliers and standardize data.
- **Zero Ratio Analysis:** Measures the proportion of zero values in a dataset, used to identify sparse features that may carry little information.
- **Group Analysis:** Examines data by dividing it into meaningful categories to compare patterns and differences across groups.
- **Hypothesis Testing:** A statistical method to determine whether there is enough evidence in data to support or reject a specific claim.
- **Variance Check:** Measures how much data values spread around the mean to understand data variability.
- **Feature Insight:** Understanding how each feature behaves and contributes to the data, used to guide feature selection and preprocessing.

Data Cleaning

- **Handling Missing Values:** Impute with mean/median/mode, or drop rows/columns with excessive missing data.
- **Handling Outliers:** Remove outliers (IQR method), Z-score method (remove $|Z| > 3$), Capping (Winsorization), Transformation (log/sqrt), or Imputation.
- **Correcting Data Types:** Ensure numeric columns are not stored as strings and vice versa.

Data Preprocessing

- **Normalization/Standardization:** MinMax Scaler scales numeric values into a fixed range (0 to 1). StandardScaler standardizes features to have mean=0 and std=1.
- **Encoding Categorical Variables:** Label Encoder converts categories into numbers. One-Hot Encoder creates separate binary columns for each category.
- **Feature Selection/Extraction:** Select the most relevant features using techniques like correlation analysis, recursive feature elimination, or PCA.
- **Data Splitting:** Split data into training, validation, and test sets (commonly 70/15/15 or 80/20 split).

Model Selection, Training & Evaluation

- **Choosing the Right Model:** Based on the problem type (classification vs regression), dataset size, and interpretability requirements.
- **Model Training:** Fit the model to the training data. Tune hyperparameters using cross-validation or grid search.
- **Model Evaluation:** Evaluate using appropriate metrics — accuracy, precision, recall, F1-score for classification; RMSE, MAE, R^2 for regression.
- **Model Deployment:** Integrate the trained model into applications and monitor its performance over time.

Linear Regression

Linear Regression is one of the simplest and most foundational supervised learning algorithms. It models the relationship between a dependent variable and one or more independent variables by fitting a linear equation to observed data.

What is Linear Regression?

Simple linear regression is a statistical method used to model the relationship between a dependent variable (Y) and an independent variable (X) by fitting a straight line through the data points. The aim is to predict the value of the dependent variable based on the independent variable using the least squares method.

The regression equation is represented as:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Here, β_0 is the intercept, β_1 is the slope of the line, and ϵ represents the error term. The slope β_1 indicates how much the dependent variable changes for a unit change in the independent variable.

Multiple linear regression extends this to multiple independent variables:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon$$

Key Concepts

- **Least Squares Method:** Minimizes the sum of squared differences between the observed values and predicted values (residuals) to find the best-fitting line.

- **R-squared (R^2):** Coefficient of determination. Measures how well the regression line fits the data. $R^2 = 1$ means a perfect fit; $R^2 = 0$ means no relationship.
- **Residuals:** The difference between actual observed values and the values predicted by the model. Residuals should be normally distributed for the model to be valid.
- **Assumptions:** Linear relationship between X and Y, independence of residuals, homoscedasticity (constant variance), and normality of residuals.

Types of Linear Regression

- **Simple Linear Regression:** One independent variable predicts one dependent variable. Models a straight-line relationship.
- **Multiple Linear Regression:** Two or more independent variables predict one dependent variable. Models a hyperplane in multi-dimensional space.
- **Polynomial Regression:** A special case where the relationship is modeled as an nth degree polynomial. Useful for non-linear data.
- **Ridge Regression (L2):** Adds an L2 penalty term to the cost function to reduce overfitting by shrinking coefficients.

```
# L2 Regularization (Ridge)
l2_model = Ridge(alpha=1.0)
l2_model.fit(X_train, y_train)
l2_pred = l2_model.predict(X_test)
print("\nL2 Coefficients:", l2_model.coef_)
print("L2 MSE:", mean_squared_error(y_test, l2_pred))
```

- **Lasso Regression (L1):** Adds an L1 penalty that can shrink some coefficients to zero, effectively performing feature selection.

```
# L1 Regularization (Lasso)
l1_model = Lasso(alpha=1.0)
l1_model.fit(X_train, y_train)
l1_pred = l1_model.predict(X_test)
print("L1 Coefficients:", l1_model.coef_)
print("L1 MSE:", mean_squared_error(y_test, l1_pred))
```

Evaluation Metrics for Regression

- **Mean Absolute Error (MAE):** The average of absolute differences between predicted and actual values. Less sensitive to outliers than MSE.
- **Mean Squared Error (MSE):** The average of squared differences between predicted and actual values. Penalizes larger errors more heavily.
- **Root Mean Squared Error (RMSE):** The square root of MSE. Has the same unit as the target variable, making it more interpretable.
- **R^2 Score:** Proportion of the variance in the dependent variable that is predictable from the independent variable(s). Range: 0 to 1.

Advantages and Disadvantages

- **Advantages:** Simple and interpretable; computationally efficient; provides statistical insights through coefficients; serves as a good baseline model.
- **Disadvantages:** Assumes a linear relationship; sensitive to outliers; can underfit complex non-linear data; requires feature scaling for regularized variants.

Python Implementation

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.datasets import load_boston # or any regression dataset
import numpy as np

# Load dataset (example with synthetic data)
import pandas as pd
from sklearn.datasets import fetch_california_housing
data = fetch_california_housing()
X, y = data.data, data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# Train model
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

# Predict and evaluate
y_pred = lr_model.predict(X_test)
print('MSE:', mean_squared_error(y_test, y_pred))
print('RMSE:', np.sqrt(mean_squared_error(y_test, y_pred)))
print('R2 Score:', r2_score(y_test, y_pred))
print('Coefficients:', lr_model.coef_)
print('Intercept:', lr_model.intercept_)
```

Key Hyperparameters

- **fit_intercept:** Whether to calculate the intercept β_0 . Default True.
- **normalize (deprecated):** Use StandardScaler from sklearn.preprocessing instead.
- **alpha (for Ridge/Lasso):** Regularization strength. Larger values specify stronger regularization.

Real-World Use Cases

- **House Price Prediction:** Predicting property prices based on features like square footage, location, and number of rooms.
- **Sales Forecasting:** Predicting future sales based on historical data, advertising spend, and seasonal trends.
- **Medical Research:** Predicting patient outcomes such as blood pressure or cholesterol levels based on patient features.
- **Stock Price Estimation:** Modeling the relationship between stock prices and economic indicators.

Logistic Regression

Logistic Regression is a supervised learning algorithm used to estimate the probability that a given instance belongs to a particular class. Despite its name, it is a classification algorithm, not a regression one.

What is Logistic Regression?

Logistic regression models the relationship between a categorical dependent variable and one or more independent variables. It is particularly useful for binary outcomes such as classifying whether an email is spam or not, whether a patient has a disease, or whether a loan will default.

Unlike linear regression, which predicts continuous numerical values, logistic regression outputs probabilities between 0 and 1. It achieves this by passing the linear combination of input features through a sigmoid (logistic) function:

$$P(Y=1) = 1 / (1 + e^{-(\beta_0 + \beta_1 X_1 + \dots + \beta_n X_n)})$$

Where p is the predicted probability, β_0 is the intercept, and $\beta_1, \dots, \beta_n$ are the coefficients for each feature. The output is then thresholded (typically at 0.5) to produce a class label.

The Sigmoid Function

The sigmoid function maps any real-valued number to a value between 0 and 1, making it ideal for probability estimation:

- **When the input is very large:** $\sigma(z) \rightarrow 1$ (predicts class 1)
- **When the input is very small (negative):** $\sigma(z) \rightarrow 0$ (predicts class 0)
- **When the input is 0:** $\sigma(0) = 0.5$ (decision boundary)

Types of Logistic Regression

- **Binary Logistic Regression:** The target variable has only two possible outcomes (0 or 1). Uses the standard sigmoid function.

- **Multinomial Logistic Regression:** The target variable has three or more unordered categories (e.g., classifying types of animals).
- **Ordinal Logistic Regression:** The target variable has three or more ordered categories (e.g., low/medium/high risk).

Loss Function and Optimization

Logistic regression uses Log Loss (Binary Cross-Entropy) as its cost function:

$$\text{Loss} = -[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$$

The model parameters are optimized using gradient descent or more advanced solvers (lbfgs, liblinear, saga) to minimize this loss function.

Evaluation Metrics for Classification

- **Accuracy:** Proportion of correctly classified instances. Best used when classes are balanced.
- **Precision:** Of all instances predicted as positive, how many are actually positive? $TP / (TP + FP)$.
- **Recall (Sensitivity):** Of all actual positives, how many were correctly identified? $TP / (TP + FN)$.
- **F1-Score:** Harmonic mean of precision and recall. Useful when there is class imbalance.
- **ROC-AUC:** Area Under the ROC Curve. Measures the model's ability to discriminate between classes.
- **Confusion Matrix:** A table showing True Positives, True Negatives, False Positives, and False Negatives.

Advantages and Disadvantages

- **Advantages:** Simple and interpretable; outputs probabilities; less prone to overfitting with regularization; works well for linearly separable data; training is fast.
- **Disadvantages:** Assumes linear relationship between features and log-odds; not suitable for complex non-linear data; sensitive to outliers; requires feature scaling.

Python Implementation

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler

# Load dataset
data = load_breast_cancer()
X, y = data.data, data.target
```

```
# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)

# Train model
log_model = LogisticRegression(C=1.0, solver='lbfgs', max_iter=1000)
log_model.fit(X_train, y_train)

# Predict and evaluate
y_pred = log_model.predict(X_test)
y_prob = log_model.predict_proba(X_test)[:, 1]
print('Accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print('Confusion Matrix:\n', confusion_matrix(y_test, y_pred))
```

Key Hyperparameters

- **C:** Inverse of regularization strength. Smaller values specify stronger regularization to prevent overfitting.
- **penalty:** Type of regularization — 'l1' (Lasso), 'l2' (Ridge, default), 'elasticnet', or 'none'.
- **solver:** Algorithm for optimization — 'lbfgs' (default), 'liblinear', 'sag', 'saga', 'newton-cg'.
- **max_iter:** Maximum number of iterations for the solver to converge. Increase if convergence warnings appear.
- **multi_class:** 'auto', 'ovr' (one-vs-rest), or 'multinomial' for multi-class problems.

Real-World Use Cases

- **Email Spam Detection:** Classifying emails as spam or not spam based on content features.
- **Medical Diagnosis:** Predicting whether a patient has a disease (e.g., cancer, diabetes) based on clinical features.
- **Credit Default Prediction:** Determining whether a borrower will default on a loan.
- **Customer Churn:** Predicting which customers are likely to stop using a service.

K-Nearest Neighbors (KNN) – Complete Guide

K-Nearest Neighbors is a simple, intuitive, non-parametric supervised learning algorithm. It makes predictions based on the similarity of a new data point to its k nearest neighbors in the training data.

What is KNN?

K-Nearest Neighbor (KNN) is a supervised learning algorithm used for both classification and regression. It is non-parametric, meaning it does not make any assumptions about the underlying data distribution, which makes it versatile for various applications. KNN works by analyzing the proximity or "closeness" of data points based on specific distance metrics.

The algorithm is also called a lazy learner because it does not build an explicit model during training. Instead, it memorizes the entire training dataset and makes predictions at query time.

How KNN Works – Step by Step

1. **Store the Training Data:** During training, KNN simply stores all training instances (no model is built).
2. **Choose K:** Select the number of nearest neighbors K to consider. This is a critical hyperparameter.
3. **Calculate Distance:** For a new test point, calculate the distance to all training points using a distance metric.
4. **Find K Nearest Neighbors:** Select the K training points with the smallest distances to the test point.
5. **Make Prediction:** For classification, predict the majority class among the K neighbors. For regression, predict the average target value.

Distance Metrics

- **Euclidean Distance:** The straight-line distance between two points in n-dimensional space. Most commonly used. Formula: $d = \sqrt{\sum (x_i - y_i)^2}$
- **Manhattan Distance:** Sum of absolute differences between coordinates. Also called L1 distance or taxicab distance.
- **Minkowski Distance:** A generalization of Euclidean and Manhattan distances. When $p=2$, it is Euclidean; when $p=1$, it is Manhattan.
- **Hamming Distance:** Used for categorical variables. Counts the number of positions where two strings differ.
- **Cosine Similarity:** Measures the angle between two vectors. Often used in text and recommendation systems.

Choosing the Right K

The choice of K is crucial for model performance:

- **Small K (e.g., K=1):** Very sensitive to noise and outliers. Low bias, high variance. Tends to overfit.
- **Large K (e.g., K=N):** Smoother decision boundaries. High bias, low variance. Tends to underfit.
- **Optimal K:** Use cross-validation to find the K that minimizes validation error. Common practice: try K from 1 to $\sqrt{N}$ and select the best.

Weighted KNN

In standard KNN, all K neighbors have equal influence. In weighted KNN, closer neighbors have more influence:

- **Distance Weighting:** Each neighbor's vote is weighted by $1/\text{distance}$, so closer neighbors contribute more to the prediction.
- Set `weights='distance'` in sklearn's `KNeighborsClassifier` to enable this.

Advantages and Disadvantages

- **Advantages:** Simple to understand and implement; no training phase; naturally handles multi-class problems; non-parametric (no assumptions about data distribution); adapts to new training data automatically.
- **Disadvantages:** Slow prediction time for large datasets ($O(n)$ per query); sensitive to irrelevant features and feature scaling; requires large memory to store training data; performance degrades in high-dimensional spaces (curse of dimensionality).

Python Implementation

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target

# Feature scaling (CRITICAL for KNN)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
                                                    random_state=42)

# Find optimal K using cross-validation
k_values = range(1, 21)
cv_scores = [cross_val_score(KNeighborsClassifier(n_neighbors=k), X_train, y_train,
                                                    cv=5).mean()
              for k in k_values]
best_k = k_values[cv_scores.index(max(cv_scores))]
print(f'Best K: {best_k}')

# Train final model
knn_model = KNeighborsClassifier(n_neighbors=best_k, weights='distance')
```

```
knn_model.fit(X_train, y_train)

# Evaluate
y_pred = knn_model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

Key Hyperparameters

- **n_neighbors (K):** Number of neighbors to use for prediction. Most important hyperparameter.
- **weights:** 'uniform' (all neighbors equal) or 'distance' (closer neighbors weighted more).
- **metric:** Distance metric to use — 'euclidean', 'manhattan', 'minkowski' (default).
- **p:** Power parameter for Minkowski distance. p=1 is Manhattan, p=2 is Euclidean.
- **algorithm:** Algorithm to compute nearest neighbors — 'ball_tree', 'kd_tree', 'brute', or 'auto'.

Real-World Use Cases

- **Recommendation Systems:** Recommending products or movies based on similarity to other users or items.
- **Image Recognition:** Classifying images based on similarity to training examples.
- **Anomaly Detection:** Identifying unusual patterns by finding data points far from their neighbors.
- **Medical Diagnosis:** Classifying patient symptoms by comparing to similar historical cases.